

Transactionality

Transactionality

Transactional query methods

By default, methods inherited from `CrudRepository` inherit the transactional configuration from [SimpleJpaRepository](#). For read operations, the transaction configuration `readOnly` flag is set to `true`. All others are configured with a plain `@Transactional` so that default transaction configuration applies. Repository methods that are backed by transactional repository fragments inherit the transactional attributes from the actual fragment method.

If you need to tweak transaction configuration for one of the methods declared in a repository, redeclare the method in your repository interface, as follows:

Example 1. Custom transaction configuration for CRUD

```
public interface UserRepository extends CrudRepository<User, Long> {  
  
    @Override  
    @Transactional(timeout = 10)  
    public List<User> findAll();  
  
    // Further query method declarations  
}
```

JAVA

Doing so causes the `findAll()` method to run with a timeout of 10 seconds and without the `readOnly` flag.

Another way to alter transactional behaviour is to use a facade or service implementation that (typically) covers more than one repository. Its purpose is to define transactional boundaries for non-CRUD operations. The following example shows how to use such a facade for more than one repository:

Example 2. Using a facade to define transactions for multiple repository calls

```
@Service  
public class UserManagementImpl implements UserManagement {
```

JAVA

```

private final UserRepository userRepository;
private final RoleRepository roleRepository;

public UserManagementImpl(UserRepository userRepository,
    RoleRepository roleRepository) {
    this.userRepository = userRepository;
    this.roleRepository = roleRepository;
}

@Transactional
public void addRoleToAllUsers(String roleName) {

    Role role = roleRepository.findByName(roleName);

    for (User user : userRepository.findAll()) {
        user.addRole(role);
        userRepository.save(user);
    }
}
}

```

This example causes call to `addRoleToAllUsers(...)` to run inside a transaction (participating in an existing one or creating a new one if none are already running). The transaction configuration at the repositories is then neglected, as the outer transaction configuration determines the actual one used. Note that you must activate `<tx:annotation-driven />` or use `@EnableTransactionManagement` explicitly to get annotation-based configuration of facades to work. This example assumes you use component scanning.

Note that the call to `save` is not strictly necessary from a JPA point of view, but should still be there in order to stay consistent to the repository abstraction offered by Spring Data.

Transactional query methods

Declared query methods (including default methods) do not get any transaction configuration applied by default. To run those methods transactionally, use `@Transactional` at the repository interface you define, as shown in the following example:

Example 3. Using `@Transactional` at query methods

```

@Transactional(readOnly = true)
interface UserRepository extends JpaRepository<User, Long> {

```

JAVA

```
List<User> findByLastname(String lastname);

@Modifying
@Transactional
@Query("delete from User u where u.active = false")
void deleteInactiveUsers();
}
```

Typically, you want the `readOnly` flag to be set to `true`, as most of the query methods only read data. In contrast to that, `deleteInactiveUsers()` makes use of the `@Modifying` annotation and overrides the transaction configuration. Thus, the method runs with the `readOnly` flag set to `false`.

NOTE

You can use transactions for read-only queries and mark them as such by setting the `readOnly` flag. Doing so does not, however, act as a check that you do not trigger a manipulating query (although some databases reject `INSERT` and `UPDATE` statements inside a read-only transaction). The `readOnly` flag is instead propagated as a hint to the underlying JDBC driver for performance optimizations. Furthermore, Spring performs some optimizations on the underlying JPA provider. For example, when used with Hibernate, the flush mode is set to `NEVER` when you configure a transaction as `readOnly`, which causes Hibernate to skip dirty checks (a noticeable improvement on large object trees).

NOTE

While examples discuss `@Transactional` usage on the repository, we generally recommend declaring transaction boundaries when starting a unit of work to ensure proper consistency and desired transaction participation.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

